

HiDiTingV100 2131E 维测功能

指导书

文档版本	00B01
发布日期	2025-06-05

前言

概述

本文档主要介绍 Hi2131V100 所支持的维测相关功能，主要为死机维测问题定位指导，以支撑对设备死机问题进行基本的问题定位。

读者对象

本文档主要适用于以下工程师：

- 软件开发工程师
- 技术支持工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。

符号	说明
	“须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 概述.....	1
2 死机维测.....	2
2.1 死机信息获取	2
2.1.1 DBK 工具 Message 界面输出	3
2.1.2 DBK 工具导出死机信息.....	3
2.1.3 连接 J-Link 调试器导出	4
2.2 死机信息内容描述	5
2.2.1 soc_syserr_head	5
2.2.2 soc_syserr_basic	5
2.2.3 soc_syserr_cmn_data.....	6
2.2.4 soc_syserr_context.....	6
2.2.5 soc_mem_info	7
2.2.6 soc_syserr_stack	7
2.2.7 soc_syserr_tail.....	8
2.3 死机问题定位	8
2.3.1 查看死机信息有效位.....	8
2.3.2 查看 main_type	9
2.3.3 看门狗重启问题定位.....	9
2.3.4 CPU 异常触发重启问题定位	10
2.3.5 liteos panic 异常定位	10
2.3.6 panic 异常定位.....	11

插图目录

图 2-1 Message 界面中输出的死机信息..... 3

表格目录

表 2-1 soc_syserr_context 成员描述 7

表 2-2 看门狗死机的关键信息..... 9

1 概述

本文档主要描述了使用单板死机维测问题定位指导，包括死机信息获取、死机信息的解析以及死机问题的定位等内容，以供 FAE 定位死机问题使用。

2 死机维测

2.1 死机信息获取

2.2 死机信息内容描述

2.3 死机问题定位

2.1 死机信息获取

死机信息可通过以下方法获取：

- 通过 DebugKits 工具的 Message 界面输出。
- 通过 DebugKits 工具的宕机信息查看功能导出。
- 连接 J-Link 调试器导出。

在研发测试实验室调试测试场景下，上述 3 种方法都适用；现网外场场景下适用 DebugKits 工具导出方法获取；各场景下连接 MCU 则可在 MCU 侧获取死机信息。

须知

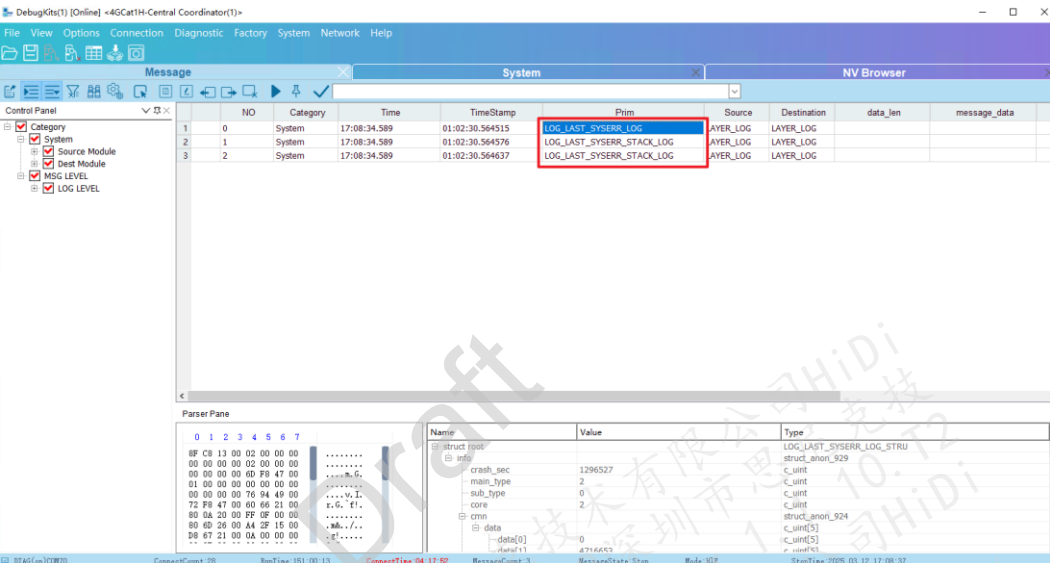
在连接 J-Link 调试器定位死机问题的过程中，为防止单板发生异常后自动重启，导致获取的死机信息无法查看，需要在定位问题之前将 A 核中死机不重启 NV 项 (NV_ID_SYSERR_UNREBOOT) 的值修改为 0x554E5253，NV 值的修改请参见《HiDiTingV100 NV 用户指南》，并重启单板。

由于单板发生异常后，不能正常重启是高风险问题，因此该功能仅可在实验室使用。

2.1.1 DBK 工具 Message 界面输出

当单板与 DebugKits 工具处于连接状态时，如果单板发生了死机，DebugKits 工具会在单板重启后进行重新连接，并将死机信息输出到 DebugKits 工具的 Message 界面中，如图 2-1 所示，死机信息将由 3 条日志组成，分别为 1 条死机原因日志（消息 ID 为 LOG_LAST_SYSERR_LOG）和 2 条死机调用栈日志（消息 ID 为 LOG_LAST_SYSERR_STACK_LOG）。

图2-1 Message 界面中输出的死机信息



2.1.2 DBK 工具导出死机信息

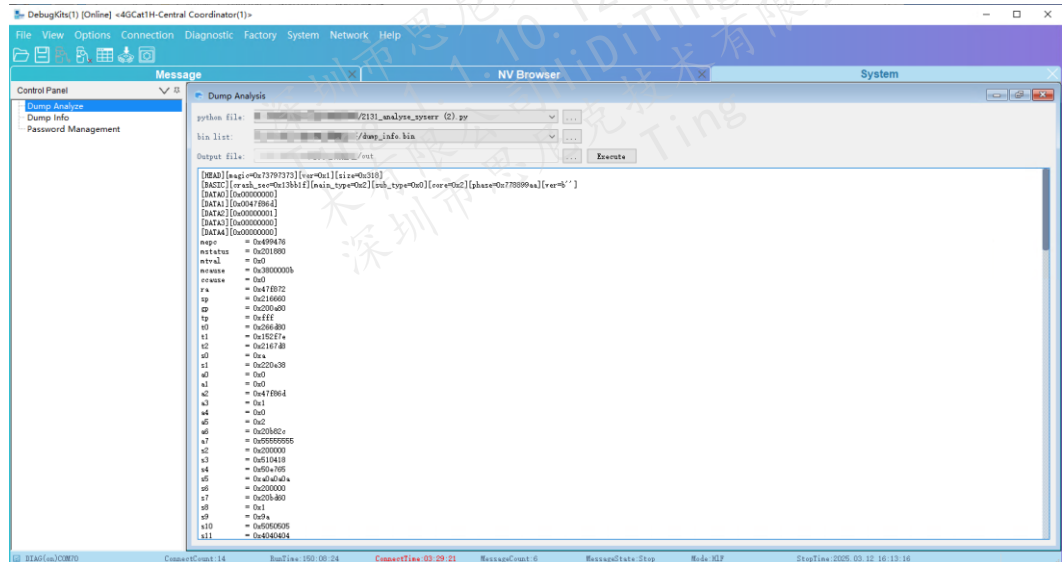
通过下方步骤，使用 DebugKits 工具导出 Flash 中的死机信息。

步骤 1 连接 DebugKits 工具，参考《HiDiTingV100 DebugKits 工具 用户指南》文档中的“导出内存数据”章节，选择 System 选项下的宕机信息查看，完成以下信息：

- 选择保存文件的路径。
- 选择导出名称。
 - soc_dump_acore_crash_memory：导出 A 核死机信息。
 - soc_dump_pcore_crash_memory：导出 C 核死机信息。
 - soc_dump_crash_flash：导出最近一次 A/C 核死机信息。

步骤 2 单击“开始 Dump(B)”按钮进行死机信息的导出。

- 步骤 3 使用“build/script/2131_analyse_syserr.py”脚本解析导出的 bin 文件，得到死机信息的 log 文件。该 log 文件中包含的死机信息可通过“2.2 死机信息内容描述”中的字段解读死机信息。



----结束

2.1.3 连接 J-Link 调试器导出

- 步骤 1 导出死机内存前，需要确保单板不会因死机导致重启，因此需要提前修改 A 核中 NV_ID_SYSTEM_SYSERR_UNREBOOT 这个 KV 项的值为 0x554E5253，配置单板在发生异常时不会重启。
- 步骤 2 确保当前环境已连接 J-Link。
- 步骤 3 在“2131_analyse_memory2.0”文件夹内使用，使用方式如下：

1. 将 map 文件放入 file 下。
A 核: application_cat1.map
C 核: protocol_cat1.map
2. 运行 main.py，根据需要选择解析 A 核或 C 核。

```
python main.py
```
3. 等待脚本执行完成后，解析文件存于 result。

common	2024/11/6 15:47	文件夹	
file	2024/11/7 14:31	文件夹	
result	2024/11/7 14:31	文件夹	
tools	2024/11/1 10:56	文件夹	
FAQ.docx	2024/11/7 14:37	Microsoft Word ...	27,636 KB
main.py	2024/11/6 15:49	Python 源文件	1 KB
README	2024/11/7 15:08	文件	1 KB

步骤 4 查看 result 目录，A 核内存解析结果查看“a_ram_result.txt”，C 核内存解析结果查看“c_ram_result.txt”。

名称	修改日期	类型	大小
a_ram_result.txt	2024/11/13 10:00	文本文档	134 KB
a_ram_simple_map.txt	2024/11/13 10:00	文本文档	631 KB
c_ram_result.txt	2024/11/18 20:37	文本文档	152 KB
c_ram_simple_map.txt	2024/11/18 21:06	文本文档	404 KB

----结束

2.2 死机信息内容描述

2.2.1 soc_syserr_head

可在生成解析结果文件中搜索以下成员名查看相关数据。

成员	描述
magic	完整有效的死机信息对应值为： EXT_SYS_ERR_START_MAGIC(0x73797373)。
ver	死机结构版本。
size	soc_syserr_info 对应的大小。

2.2.2 soc_syserr_basic

可在生成解析结果文件中搜索以下成员名查看相关数据。

成员	描述
crash_sec	死机发生的时间（此处为距离上电后的绝对时间，包含深睡眠时间）。
ver	死机发生时的 image 版本号。暂未记录。

成员	描述
main_type	死机的主类型： - SOC_SYSERR_WATCH_DOG(1)：看门狗到期触发的重启。详细内容请参见“2.3.3 看门狗重启问题定位”章节。 - SOC_SYSERR_PANIC(2)：主动调用 panic 的重启。 - SOC_SYSERR_CPU_EXEC(3)：CPU 异常触发的重启。 - SOC_SYSERR_OS_ERR(4)：OS 异常重启，暂未使用。 - SOC_SYSERR_LOS_PANIC(5)：OS 异常触发的重启。
sub_type	死机发生子类型（当 main_type 为 5 时，可表示父函数地址、任务名称、任务 ID 等内容，需要结合其他死机信息综合判断）。
core	死机发生的核。 1：C 核； 2：A 核。
phase	死机信息记录的存储阶段。存储死机过程中可能发生 2 次异常，从而导致死机信息存储不完整（例如，踩内存场景，遍历内存池造成 2 次死机），当该值为 SOC_SYSERR_SAVE_PHASE_FINISH (0x778899AA) 时，表示进行了完整的死机存储。

2.2.3 soc_syserr_cmnn_data

可在生成解析结果文件中搜索以下成员名查看相关数据。

成员	描述
data[4]	不同的崩溃场景存储不同的信息，常用于 panic 定位。

2.2.4 soc_syserr_context

soc_syserr_context 中记录了死机时的 CPU 状态，[表 2-1](#) 中仅对具有特殊功能的成员进行描述，且只有在 SOC_SYSERR_CPU_EXEC 和 SOC_SYSERR_WATCH_DOG 场景下意义准确。可在生成解析结果文件中搜索以下成员名查看相关数据。

表2-1 soc_syserr_context 成员描述

成员	描述
mepc	死机发生时的 pc 值。
ra	调用死机发生函数的地址（返回地址）。例如，死机发生在 A 函数，在 x 地址调用的 A 函数，则 ra 为 x 指令的下一条指令。
sp	死机发生时的栈指针。
mstatus	死机发生时的 CPU 状态寄存器。
mtval	死机发生时程序预访问但 CPU 和逻辑限制访问的目标地址，Load、store 异常时，该值有意义。
mcause	死机原因，已经转换为 sub_type。
ccause	死机原因，CPU 子类。

2.2.5 soc_mem_info

死机发生时的内存池状态。mem_info 的具体成员信息请参见头文件 “soc_mem.h” 中的 “soc_mem_info” 结构体定义。

2.2.6 soc_syserr_stack

可在生成解析结果文件中搜索以下成员名查看相关数据。

成员	描述
in_isr	死机是否发生在中断中。
sp_in_stack	死机发生时的 sp 是否在合法栈空间中。
id	死机发生时的任务 ID。即保存当前线程任务执行状态的地址，具体可参考 LosTaskCB 结构体。
top	栈上边界。
bottom	栈下边界。
sp	栈指针。
Peek	栈峰值。死机发生在任务中时，该值有意义。

成员	描述
Name	发生死机的栈的名称。
Stack	死机时刻的函数调用栈。

2.2.7 soc_syserr_tail

可在生成解析结果文件中搜索以下成员名查看相关数据。

成员	描述
magic	完整有效的死机信息对应该值为：EXT_SYS_ERR_END_MAGIC (0x73797365) 。

2.3 死机问题定位

2.3.1 查看死机信息有效位

成员	描述
soc_syserr_head[magic]	EXT_SYS_ERR_START_MAGIC (0x73797373)：死机信息有效。
soc_syserr_tail[magic]	EXT_SYS_ERR_END_MAGIC (0x73797365)：死机信息有效。
soc_syserr_basic[phase]	- SOC_SYSERR_SAVE_PHASE_HEAD_TAIL (0x01)：soc_syserr_head 值有效。 - SOC_SYSERR_SAVE_PHASE_BASIC (0x02)：soc_syserr_basic 及以上的值有效。 - SOC_SYSERR_SAVE_PHASE_CMN (0x03)：soc_syserr_cmndata 及以上的值有效。 - SOC_SYSERR_SAVE_PHASE_CPU_CONTEXT (0x04)：soc_syserr_context 及以上的值有效。 - SOC_SYSERR_SAVE_PHASE_MEM_INFO (0x05)：

成员	描述
	soc_mem_info 及以上的值有效。 • SOC_SYSERR_SAVE_PHASE_SYS_STATUS (0x06) : soc_syserr_sys_status 及以上的值有效。 • SOC_SYSERR_SAVE_PHASE_STACK (0x07) : soc_syserr_stack 及以上的值有效。 • SOC_SYSERR_SAVE_PHASE_FINISH (0x778899AA) : 死机信息完整有效。

2.3.2 查看 main_type

通过查看死机信息中的 main_type 字段，判断当前死机的主类型，确定定位方向。

成员	描述
main_type	• SOC_SYSERR_WATCH_DOG(1): 看门狗到期触发的重启。详细内容请参见“2.3.3 看门狗重启问题定位”章节。 • SOC_SYSERR_PANIC(2): 主动调用 panic 重启。 • SOC_SYSERR_CPU_EXEC(3): CPU 异常触发重启。 • SOC_SYSERR_OS_ERR(4): OS 异常触发的重启，暂未使用。 • SOC_SYSERR_LOS_PANIC(5): OS 异常触发重启。

2.3.3 看门狗重启问题定位

看门狗死机导致的重启问题，大概率为代码中存在死循环（包含次数很大的循环）或某个业务一直被执行（如灌包），使得 IDLE 任务无法及时得到调度，在规定时间内没有进行踢狗而导致看门狗超时造成的重启。通常可以通过[表 2-2](#)中的信息并结合 lst 文件确定具体的死机位置。

表2-2 看门狗死机的关键信息

成员	描述
mepc	通过 mepc 确定看门狗到期时的 pc。根据概率推测可知，该代码基本为死循环中会调用的代码。

成员	描述
ra 和栈	如果 mepc 在通用函数中，如 memcpy 函数中，可以通过 ra 和栈内容进一步确认函数调用流程。

2.3.4 CPU 异常触发重启问题定位

通过 sub_type 成员描述内容，可确认 CPU 异常触发重启具体的死机原因。

成员	描述
sub_type	0: Instruction address misaligned, 取指地址未对齐, riscv 要求指令在双字节对齐位置。 1: Instruction access fault, 取指地址异常。 2: Illegal instruction, 非法指令。 4: Load address misaligned, 读取数据的目标地址未对齐。 5: Load access fault, 读取数据的目的地址禁止进行读操作。 6: Store/AMO address misaligned, 存储数据的目标地址未对齐。 7: Store/AMO access fault, 存储数据的目的地址禁止进行写操作。

2.3.5 liteos panic 异常定位

成员	描述
sub_type	存储 reson、LiteOS panic 死机的子类型。 - EXT_PANIC_MALLOC_FAIL (110) : 内存分配失败。 - EXT_PANIC_MUTEX_ERROR (111) : 创建/获取/释放互斥锁失败。
cmn.data[0]~cmn.data[4]	存储死机相关信息，如父函数地址、任务名称、任务 ID 等内容，需要结合其他死机信息综合判断。

2.3.6 panic 异常定位

调用 void panic(panic_id_t source, uint32_t code)函数触发的异常。

成员	描述
sub_type	存储触发的 panic 的模块 id (panic_id_t) , 可参考 “protocol/cat1/lte/infra/comm/inc/panic.h” 内部 panic_id 定义, panic 死机的子类型。 • EXT_PANIC_ADAPT_OLD_PANIC (112) : 产品内部独有的 panic 类型。 • EXT_PANIC_PD_MALLOC_FAIL (114) : 调用以 panicing_sw_pd 或 panicing_hw_pd 结尾的内存接口, 且内存申请失败。 • EXT_PANIC_DOUBLE_FREE (115) : 内存重复释放。 • EXT_PANIC_IPC_TIME_OUT (116) : C 核发送 IPC 消息超时。 • EXT_PANIC_STACK_PROTECT_GUARD (120) : A 核栈被踩。 • EXT_PANIC_NV_INIT_FAIL (122) : NV 分区信息获取失败。 • EXT_PANIC_EXCEED_MAX_TASK_NUM (130) : C 核创建的任务数量超出最大限制。